Modelos fundacionales y LLM

Introducción

Cuando alguien usa por primera vez un sistema como ChatGPT, Claude o Gemini, la experiencia suele generar dos reacciones casi simultáneas: asombro ante lo que puede hacer y desconcierto ante cómo falla. El mismo sistema que sintetiza un informe técnico de cien páginas en tres párrafos precisos puede, en la siguiente consulta, afirmar con total confianza un dato que es completamente falso. El mismo que genera código funcional en segundos puede equivocarse en un cálculo aritmético elemental. Esa asimetría no es aleatoria ni arbitraria. Tiene una explicación, y entenderla requiere ir por dentro del sistema.

Estos modelos no buscan información en una base de datos ni siguen reglas codificadas. Aprenden patrones estadísticos a partir de enormes volúmenes de texto y generan respuestas que son estadísticamente posibles dado el contexto. Esa diferencia, entre recuperar información verificada y generar texto probable, tiene consecuencias prácticas que no son evidentes hasta que se comprende la arquitectura subyacente.

En los últimos años emergió un tipo particular de sistema que concentra y amplifica esas capacidades. Bommasani, Hudson, Adeli, Altman, Arora, Koreeda y Liang, (2021) lo llamaron modelo fundacional: son sistemas entrenados sobre cantidades masivas de datos variados que pueden adaptarse a una enorme diversidad de tareas sin ser rediseñados para cada una. Dentro de esa familia, los modelos de lenguaje de gran escala o LLM (*Large Language Model System*) son los que más atención han recibido y los que más impacto tienen hoy en contextos profesionales concretos.

Para quien trabaja en ciencia de datos, un LLM puede clasificar documentos, generar código de análisis o sintetizar reportes extensos. Para quien gestiona seguridad informática, puede leer *logs*,

explicar alertas técnicas y redactar informes de incidentes. Para quien desarrolla productos, puede prototipar flujos conversacionales en horas. Pero ninguna de esas aplicaciones funciona bien si no se entiende cuándo confiar en el sistema, cuándo verificarlo y cuándo compensar sus limitaciones con herramientas externas.

Por eso, el recorrido de esta lectura intentará responder las preguntas bases: ¿Qué son los modelos fundacionales y qué los distingue de los modelos clásicos?, ¿cómo funciona la arquitectura *Transformer* que está detrás de cada LLM?, ¿qué significa el *prompting* y por qué pequeñas variaciones en cómo se formula una instrucción pueden cambiar? y ¿cómo herramientas como el RAG o las API externas extienden lo que estos sistemas pueden hacer en el mundo real?

Caso: CNET y la experiencia con artículos generados por IA

A partir de noviembre de 2022, el medio tecnológico estadounidense CNET comenzó a publicar artículos de finanzas personales generados por una herramienta de inteligencia artificial (IA) desarrollada internamente. El único indicio de su origen automatizado era un texto visible solo al pasar el cursor sobre la firma: producido usando tecnología de automatización. En dos meses, el medio publicó 77 artículos bajo ese esquema, cubriendo temas como interés compuesto, cuentas de ahorro, préstamos hipotecarios y certificados de depósito.

El 12 de enero de 2023, el medio de investigación tecnológica Futurism reveló públicamente que CNET había estado publicando contenido generado por IA sin una divulgación prominente. Lo que convirtió el caso en un hito no fue solo la falta de transparencia, sino lo que el análisis de los artículos reveló. Se identificaron al menos cinco errores (Futurism, 2023).

El más llamativo fue que el sistema afirmaba que depositar $10.000 en una cuenta de ahorro con una tasa de interés del 3% anual generaría ganancias de $10.300 al final del primer año. El cálculo correcto es que el interés ganado sería $300; los $10.300 representan el saldo total, incluyendo el capital inicial, no las ganancias. El mismo artículo confundía las frecuencias de capitalización de los certificados de depósito y mezclaba conceptos.

La editora en jefe de CNET, Connie Guglielmo, publicó un editorial el 25 de enero de 2023 en el que reconoció la situación e informó que el medio había conducido una auditoría interna de los 77 artículos. El resultado fue que 41 de ellos, es decir, más de la mitad, requirieron correcciones. CNET pausó la publicación de artículos generados por IA mientras revisaba sus procesos.

Lo que hizo el caso especialmente instructivo fue la asimetría de los errores. No todos los artículos fallaban de la misma manera ni con la misma frecuencia. Algunos requerían correcciones sustanciales; otros solo ajustes menores; y un grupo pasó la auditoría sin cambios. El sistema producía resultados de calidad muy distinta según el tema y el nivel de especificidad requerido, sin que esa variación fuera predecible para los editores que revisaban el contenido.

Además, todos los artículos habían pasado por revisión humana antes de publicarse. Los editores los leían, los verificaban y los aprobaban. Aun así, los errores habían escapado a ese proceso, en parte porque la fluidez del texto hacía más difícil la lectura crítica. Cuando algo suena autorizado y bien redactado, el cerebro humano tiende a leerlo con menos escepticismo.

El caso de CNET no es una historia sobre un medio irresponsable. Es una puerta de entrada a las preguntas centrales que esta lectura busca responder: ¿por qué un sistema puede redactar con fluidez impecable y calcular mal un interés simple? ¿Por qué sus errores son tan difíciles de detectar? ¿Por qué rinde de forma desigual según el tema? ¿Y qué dice eso sobre cómo está construido por dentro?

Reflexiona

> Antes de continuar, considera lo siguiente: el sistema que usó CNET revisaba artículos de finanzas personales, un dominio donde los errores tienen consecuencias directas para los lectores. El texto era fluido, la estructura era correcta y los editores humanos que revisaban los artículos no detectaron los errores antes de la publicación.

¿Por qué un sistema capaz de redactar un artículo coherente de mil palabras calcula mal cuánto rinde una cuenta de ahorro? ¿Tiene el sistema alguna forma de saber que se equivocó? ¿Y qué diferencia hay entre este sistema y un motor de búsqueda que recupera información de fuentes verificadas?

Modelos fundacionales

El término fue propuesto por Bommasani et al. (2021) para describir un tipo de sistema de aprendizaje automático que comparte tres características definitorias:

1. Está entrenado en cantidades masivas de datos variados.
2. Puede adaptarse a una enorme diversidad de tareas posteriores.
3. Constituye una base sobre la cual se construyen sistemas más especializados.

La metáfora de fundación es deliberada, ya que estos modelos son la infraestructura sobre la que se apoya una nueva generación de aplicaciones inteligentes.

Un clasificador entrenado para detectar fraudes en transacciones de tarjetas de crédito es excelente en esa tarea y prácticamente inútil fuera de ella. Un *foundation model*, en cambio, ha absorbido patrones lo suficientemente generales como para redactar informes financieros, analizar el sentimiento de los clientes en redes sociales, automatizar respuestas de soporte técnico y resumir contratos comerciales, aunque no fue optimizado explícitamente para ninguna de esas tareas en particular.

Russell y Norvig (2016) describían el aprendizaje como la capacidad de un agente de modificar su comportamiento en función de la experiencia acumulada. Los *foundation models* llevan ese

principio a una escala donde la experiencia acumulada es una fracción significativa del conocimiento escrito de la humanidad.

Este cambio de paradigma tiene consecuencias prácticas inmediatas. En el modelo tradicional, construir un sistema inteligente para un dominio nuevo requería recolectar datos etiquetados, diseñar características relevantes, entrenar un modelo desde cero y evaluarlo exhaustivamente, todo ello para una sola tarea. Con los *foundation models*, ese proceso se comprime radicalmente: se parte de una base ya entrenada y se la adapta.

El esfuerzo se desplaza desde la ingeniería del modelo hacia la ingeniería de la interacción con el modelo, lo que abre la puerta a que profesionales de dominios muy distintos (médicos, abogados, biólogos, ingenieros, etc.) puedan construir aplicaciones sin necesidad de ser expertos en aprendizaje automático.

Familia de modelos fundacionales

No todos los *foundation models* trabajan con texto. La idea de entrenar un modelo masivo sobre grandes volúmenes de datos y luego adaptarlo es aplicable a cualquier tipo de dato, y eso ha dado lugar a una familia amplia de sistemas. Entender esta taxonomía es importante porque normalmente se trabaja con modelos que combinan más de una modalidad.

Los LLM son el subconjunto más conocido. Es un sistema de inteligencia artificial entrenado sobre enormes volúmenes de texto con el objetivo de aprender las regularidades estadísticas del lenguaje: qué palabras tienden a seguir a otras, qué estructuras son coherentes, cómo se relacionan los conceptos.

A partir de ese entrenamiento, el modelo es capaz de generar texto nuevo, responder preguntas, razonar sobre problemas y producir código, traducciones

Los modelos de visión y lenguaje (VLM, *Vision-Language Model*) integran la comprensión de imágenes con el procesamiento de texto. Los sistemas como Gemini Omni y GPT-4o son ejemplos claros, ya que no solo pueden describir una imagen o responder preguntas complejas sobre ella, sino que procesan video en tiempo real, comparan múltiples fotografías o extraen datos estructurados de gráficos complejos.

o resúmenes, entre muchas otras tareas. Los modelos como GPT-4, Claude o Gemini fueron entrenados con millones de parámetros sobre datos que abarcan buena parte del conocimiento escrito disponible en internet, libros y código fuente.

Los modelos de audio procesan señales sonoras tales como transcripción de voz a texto, identificación de idiomas, separación de fuentes de audio. Whisper, desarrollado por OpenAI, es el ejemplo más extendido. En robótica, estos modelos permiten que los robots reciban instrucciones verbales en entornos ruidosos. En aplicaciones de innovación, habilitan interfaces conversacionales más naturales.

Los modelos generativos de imagen y video aprenden representaciones visuales a partir de enormes colecciones de imágenes y sus descripciones textuales. Midjourney v6 o Stable Diffusion 3 / Flux son ejemplos actuales. Aunque su mecanismo interno difiere en algunos aspectos del de los LLM, comparten la filosofía fundacional de realizar entrenamiento masivo sobre datos variados, seguido de adaptación a tareas específicas.

Los modelos multimodales de propósito general, como Gemini 1.5 Pro o Claude 3.5 Sonnet, representan la siguiente frontera, un único modelo capaz de razonar sobre texto, imágenes, audio y código de forma integrada. Bommasani et al. (2021) anticiparon esta evolución y documentan sus implicaciones, ya que al integrar

múltiples modalidades en una única arquitectura, aparece la posibilidad de razonar sobre el mundo de formas que ninguna modalidad aislada permite.

Los LLM como caso particular

Los modelos de lenguaje son, dentro de los *foundation models*, los que más atención han recibido y los que más impacto tienen hoy en contextos profesionales concretos.

La idea detrás de su entrenamiento es que, dado un fragmento de texto, se busca predecir cuál es la siguiente palabra. Pero esa tarea aparentemente simple obliga al modelo a aprender gramática, lógica, conocimiento factual, estilos de razonamiento. No porque alguien lo haya diseñado así, sino porque sin todo eso, la predicción falla. Es un efecto secundario del proceso de optimización, no un objetivo declarado.

Lo que Murphy (2022) llama aprendizaje de representaciones captura bien esto. El modelo construye su propio mapa interno del lenguaje, sin que nadie le indique qué estructuras buscar. GPT-3 se entrenó con 570 GB de texto y 175 mil millones de parámetros. Sus sucesores son más grandes, aunque los detalles exactos rara vez se hacen públicos.

FENÓMENO DE LA EMERGENCIA

Una de las propiedades más sorprendentes de los LLM es que ciertas capacidades no aparecen de forma gradual a medida que el modelo crece, sino de golpe, al cruzar ciertos umbrales de escala. Bommasani et al. (2021) documentaron este fenómeno y lo llamaron emergencia: habilidades como el razonamiento aritmético, la generación de código, la traducción en idiomas poco frecuentes o la inferencia causal pueden estar completamente ausentes en un modelo de 10 mil millones de parámetros y aparecer sin dificultad en uno de 100 mil millones, sin que nadie las haya diseñado explícitamente.

Esto tiene una consecuencia práctica importante y es que los LLM son genuinamente difíciles de predecir, incluso para quienes los construyen. No alcanza con extrapolar el comportamiento de un modelo pequeño para anticipar qué va a poder hacer uno más grande. Esa imprevisibilidad es tanto una limitación como una fuente de sorpresas útiles para quienes los usan en contextos profesionales.

Experimenta

Explora cómo un mismo *foundation model* se adapta a problemas de diferentes disciplinas, sin cambiar su arquitectura ni sus pesos.

```
Prompt sugerido: Actúa como un experto en [tu carrera]. Explícame en tres párrafos cómo los foundation models como GPT-4 están transformando el trabajo en mi disciplina. Usa ejemplos concretos de herramientas o aplicaciones reales de 2022-2025.
```

Luego repite el mismo *prompt* cambiando únicamente la carrera mencionada y compara las respuestas.

> **Reflexión:** ¿En qué aspectos la respuesta para tu carrera difiere de lo que respondería para otra? ¿El modelo 'sabe' igual de bien sobre todas las carreras, o hay diferencias en la profundidad y precisión de las respuestas?

GENERALIZACIÓN Y TRANSFERENCIA

La transferencia del conocimiento aprendido a nuevas tareas es quizás la propiedad más importante de los *foundation models*. En aprendizaje automático clásico, la generalización hace referencia a que un modelo funcione bien con datos nuevos del mismo tipo que los de entrenamiento. Los *foundation models* exhiben una forma de generalización más ambiciosa, ya

que funcionan razonablemente bien en tareas que no formaron parte del entrenamiento en absoluto.

Raschka y Mirjalili (2017) describen el *transfer learning* como una de las técnicas más poderosas del aprendizaje automático moderno. En lugar de inicializar los pesos de una red de forma aleatoria, se parte de pesos preentrenados que ya capturan características relevantes. Los *foundation models* llevan este principio al extremo; los pesos iniciales son el resultado de entrenamiento sobre prácticamente todo el texto disponible en Internet, lo que los convierte en puntos de partida para cualquier tarea.

En ciencia de datos, por ejemplo, alcanza con unos pocos ejemplos etiquetados para que un modelo clasifique documentos financieros de una empresa en particular, algo que antes exigía cientos de ejemplos y semanas de trabajo manual. Y la misma base sirve para análisis de sentimiento, extracción de entidades, resúmenes, generación de reportes.

En seguridad informática pasa algo parecido. Se ha demostrado que los LLM pueden detectar vulnerabilidades en código con solo mostrarles algunos ejemplos de patrones problemáticos como contexto, sin necesidad de reentrenar nada. También se usan para leer *logs*, explicar alertas técnicas en términos que cualquiera pueda entender o redactar reportes de incidentes.

En robótica, los *foundation models* funcionan como un módulo de comprensión que permite al robot entender instrucciones en lenguaje natural y traducirlas a acciones concretas. En lugar de programar cada comportamiento posible, el robot puede recibir una orden como 'lleva esto a la mesa de la izquierda' y encontrar solo la manera de ejecutarla.

Arquitectura transformer

Detrás de cada LLM hay una arquitectura específica que hace posible todo lo que has visto hasta aquí. El *transformer*, presentado en el influyente *paper Attention is All You Need* de Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser y Polosukhin (2017) es esa arquitectura. Casi una década después, sigue siendo la base de prácticamente todos los modelos de lenguaje de gran escala. Comprender sus componentes conceptuales es indispensable para cualquier profesional que trabaje con estas tecnologías.

Tokenización y embeddings

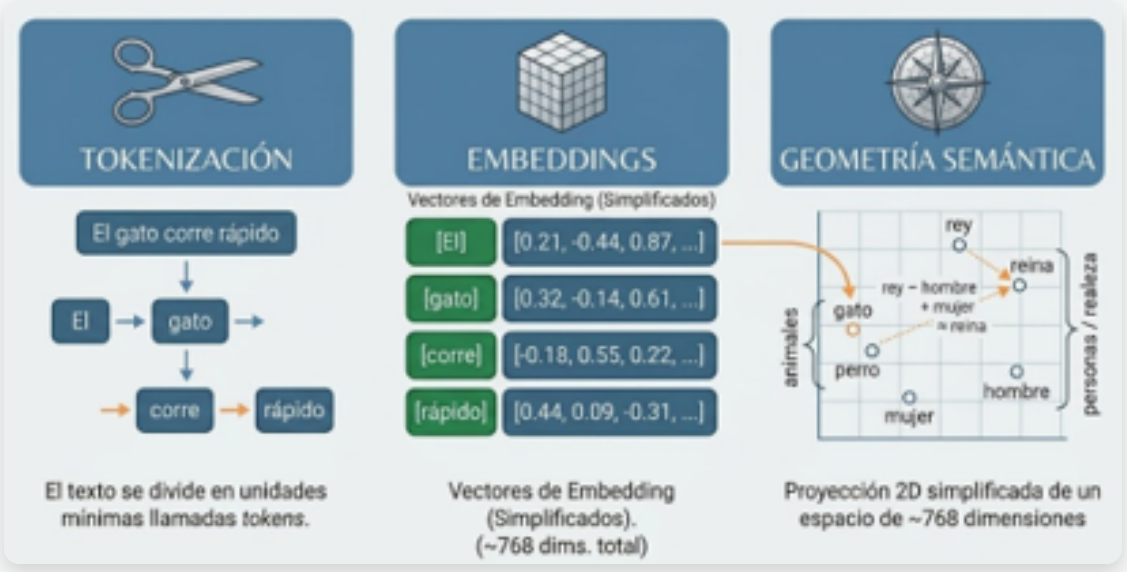
Todo empieza con un problema básico y es que las computadoras no entienden texto, entienden números. Entonces, lo primero que hace un LLM es dividir la oración en fragmentos llamados *tokens*. "El gato corre rápido" se divide en piezas como [El], [gato], [corre], [rápido].

Hasta ahí, solo fragmentos. El paso siguiente es convertir cada *token* en un vector: una lista de cientos de números que representa a esa palabra en un espacio matemático. Esa representación se llama *embedding*, y es lo que el modelo realmente procesa. No es una conversión manual ni una tabla fija: el modelo ajusta esos vectores millones de veces durante el entrenamiento hasta que *tokens* con significados parecidos terminen con *embeddings* parecidos. Lo que hace interesantes a esos vectores no es que existan, sino dónde quedan ubicados, porque forman una especie de mapa donde la distancia entre palabras refleja qué tan relacionadas están semánticamente.

Y de esa geometría emerge algo que nadie diseñó explícitamente. Si tomas el *embedding* de 'rey', restas el de 'hombre' y sumas el de 'mujer', el resultado cae justo cerca del *embedding* de 'reina'. Esa relación no fue programada, ya que surgió sola del entrenamiento, como un patrón que el modelo descubrió por exposición masiva al lenguaje.

Un científico de datos puede usar esos *embeddings* directamente para comparar documentos, agruparlos por tema o detectar cuáles hablan de lo mismo aunque usen palabras distintas. Alguien en seguridad puede identificar mensajes de *phishing* aunque el texto cambie, porque la intención queda capturada en las coordenadas del vector. En bioinformática se usa el mismo principio para analizar proteínas; aminoácidos que aparecen en contextos similares terminan con *embeddings* cercanos, revelando patrones funcionales sin que nadie los haya etiquetado.

Figura 1. Tokenización y embeddings



Fuente: elaboración propia.

El mecanismo de atención

La palabra 'banco' puede ser un mueble o una institución financiera. Para un humano, el contexto lo resuelve sin esfuerzo. El mecanismo de atención le da al modelo esa misma capacidad: cuando procesa una palabra, no la analiza de forma aislada, sino que mira todas las demás y decide cuánto peso darle a cada una.

Figura 2. Mecanismo de atención



Fuente: elaboración propia.

Como puedes observar en la figura 2, en 'Me senté en el banco del parque', las palabras 'senté' y 'parque' reciben mucha atención al procesar 'banco'. En 'Pedí un préstamo en el banco', son 'préstamo' y 'banco' las que pesan. El resultado es que la misma palabra termina con representaciones distintas según lo que la rodea, lo cual captura exactamente cómo funciona el lenguaje natural.

Técnicamente, el modelo aprende tres vectores por cada *token*: una **query** (qué está buscando ese *token* en el contexto), una **key** (qué ofrece al contexto) y un **value** (qué información aporta si es seleccionado). La atención entre dos *tokens* se calcula comparando la **query** de uno con la **key** del otro. Si se parecen, hay mucha atención. Si no, poca. Y esto sucede en paralelo para todos los pares a la vez, lo que permite capturar relaciones de largo alcance en una sola pasada.

En traducción automática, esto conecta palabras en el idioma fuente con sus equivalentes en el destino, aunque el orden cambie. En generación de código, mantiene consistencia entre la declaración de una variable y su uso veinte líneas después. En análisis de seguridad, conecta indicadores dispersos a lo largo de un informe técnico y construye una imagen coherente del incidente.

Todo eso, la tokenización, los *embeddings*, la atención, ocurre antes de que el modelo produzca una sola palabra. Son los mecanismos que le permiten entender el contexto. Lo que viene después es la generación, es decir, cómo el modelo usa todo eso para decidir, paso a paso, qué escribir a continuación.

Generación de contenido

Cuando el modelo está por escribir la próxima palabra, el mecanismo de atención ya hizo su trabajo: pesó cada *token* del historial y determinó cuáles son más relevantes para lo que viene. Ese resultado no queda guardado en ningún lado; se convierte directamente en una distribución de probabilidades sobre el vocabulario completo. Por eso, después de 'El gato...', 'come' aparece con 70% de probabilidad, 'duerme' con 20% y 'salta' con 10%. No es una regla fija ni una tabla de frecuencias; es la consecuencia directa de cuánta atención recibió cada parte del contexto.

Pero el modelo no siempre elige la palabra más probable. Hay un parámetro llamado temperatura que controla cuánto riesgo toma:

- Con temperatura baja, el modelo es conservador; casi siempre elige la opción más probable y el resultado es predecible, pero consistente.
- Con temperatura alta, le da más peso a opciones menos probables, lo que produce textos más variados y creativos, pero también más propensos a desviarse.

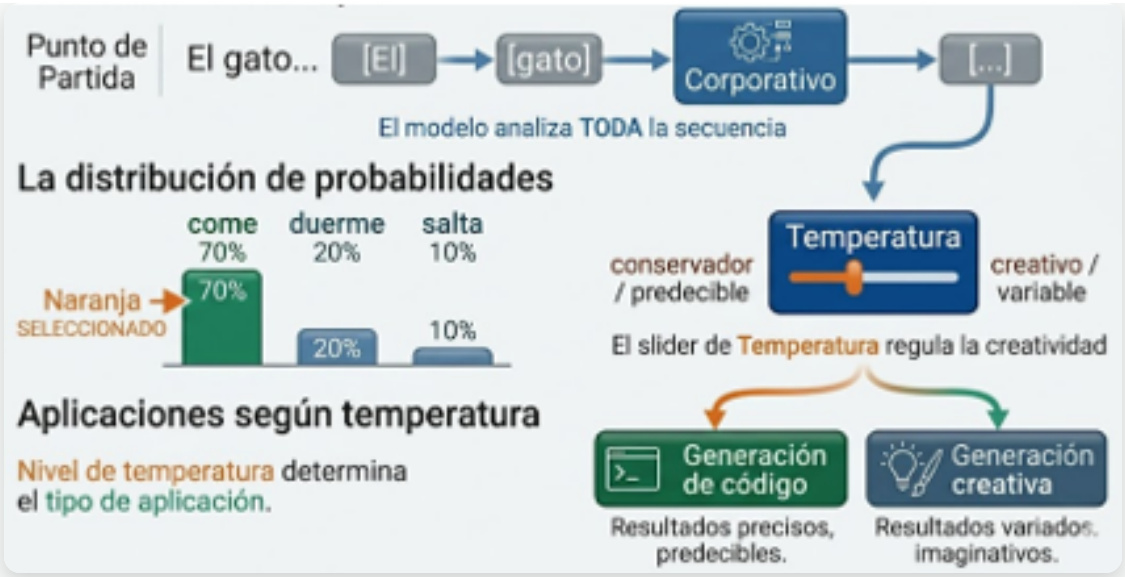
Un asistente de código generalmente corre con temperatura baja. Un generador de ideas creativas, con temperatura más alta.

En la práctica, los *transformers* utilizan múltiples cabezas de atención que operan en paralelo, cada una capturando un tipo distinto de relación: algunas conectan sujeto con verbo, otras resuelven referencias entre pronombres y sustantivos, otras capturan relaciones semánticas más abstractas. El resultado final combina todas esas perspectivas.

No hay memoria explícita ni base de datos de hechos, sino que hay atención sobre toda la secuencia, en cada paso de la generación. Para el modelo, el código es texto, ya que se tokeniza y se procesa exactamente igual, pero el entrenamiento con millones de líneas de código le permitió aprender su sintaxis y sus convenciones como un patrón más del lenguaje.

Eso es lo que permite mantener coherencia a lo largo de un texto largo. En la redacción de informes, el modelo puede sostener el hilo del argumento durante páginas sin perder el foco inicial. En generación de código, mantiene consistencia entre la declaración de una variable y su uso veinte líneas después. En análisis de documentos de seguridad, conecta indicadores dispersos a lo largo de un informe técnico y construye una imagen coherente del incidente a partir de evidencia fragmentada.

Figura 3. Generación de contenido



Fuente: elaboración propia.

Procesamiento paralelo

Una diferencia fundamental entre *transformer* y sus predecesores (las redes recurrentes o LSTM) es el procesamiento paralelo. Las redes recurrentes procesaban el texto de forma secuencial, es decir, primero el *token* 1, luego el 2, luego el 3. Cada paso dependía del anterior, lo que hacía el entrenamiento lento. El *transformer*, en cambio, procesa todos los *tokens* de la secuencia de forma simultánea, lo que permite aprovechar la potencia de los GPU (*Graphics Processing Unit*) modernos de forma masiva.

Raschka y Mirjalili (2017) destacan que la eficiencia computacional no es un detalle de implementación, sino una condición, porque sin el procesamiento paralelo, el entrenamiento de modelos con cientos de miles de millones de parámetros habría sido demasiado lento con la infraestructura disponible. La elección arquitectónica tuvo, por lo tanto, consecuencias históricas.

La ventana de contexto y sus implicancias prácticas

> Una de las propiedades más importantes de un LLM es la ventana de contexto, que indica la cantidad máxima de *tokens* que el modelo puede procesar simultáneamente. Los primeros modelos tenían ventanas de 512 o 2048 *tokens*. Los modelos recientes como Gemini Pro tienen ventanas de hasta un millón de *tokens*, lo que permite procesar libros completos o bases de código extensas en una sola llamada.

Eso cambia bastante cómo se trabaja con estos sistemas en contextos reales. Si un analista debe procesar un informe de 200 páginas, con una ventana chica debe elegir qué fragmentos enviar y ya está tomando decisiones editoriales antes de que el modelo haga nada. Un especialista en seguridad que revisa *logs* extensos tiene el mismo problema, porque no puede enviar el archivo completo, tiene que intuir qué parte contiene lo relevante.

Aunque el modelo procesa toda la ventana de contexto de forma paralela durante el entrenamiento, la generación de respuestas sí es secuencial, *token* por *token*. El modelo genera un *token*, después el siguiente condicionado por el anterior y así sucesivamente.

Cada palabra está entonces condicionada por todo lo anterior, incluyendo el *prompt* original y los *tokens* que ya generó. Esto explica la coherencia en textos largos, pero también explica algo que cualquiera que use estos modelos seguido habrá notado: en conversaciones muy extensas o respuestas muy largas, el modelo puede perder el hilo.

No es que olvide literalmente, sino que cuando el contexto está casi lleno, las instrucciones del principio quedan más lejos y tienen menos peso relativo. Si se le pide que escriba en primera persona, a las diez interacciones volvió a la tercera. Si se le da una restricción importante en el primer mensaje, dos horas después ya no la está respetando. No es un bug exactamente, es una consecuencia de cómo funciona la atención: no todo lo que está en la ventana tiene el mismo peso.

Reflexiona

Antes de continuar con la siguiente sección, tómate unos minutos para responder estas preguntas:

1. ¿Cuál de las propiedades de los LLM vistas hasta aquí (emergencia, transferencia, atención, ventana de contexto) te parece más relevante para los problemas típicos de tu disciplina? ¿Por qué?

2. Considera una tarea concreta que se realiza en tu campo profesional. ¿Qué parte de esa tarea podría beneficiarse de un LLM?

3. ¿Qué parte de esa misma tarea no podría beneficiarse de un LLM y por qué?

Prompting

Si los *foundation models* son la infraestructura y los *transformers* son el motor, el *prompting* es la interfaz. Es la habilidad de comunicarse efectivamente con un LLM; formular instrucciones, proporcionar contexto, estructurar la tarea y controlar el formato de la respuesta. Pequeñas variaciones en cómo se formula una pregunta pueden producir diferencias importantes en la calidad de la respuesta.

Wang, Ma, Feng y Zhang (2024) documentan que el diseño del *prompt* es una de las habilidades más importantes a desarrollar para obtener resultados útiles de los LLM en aplicaciones reales. No es exagerado afirmar que el *prompting* efectivo es una forma de programación; en lugar de especificar pasos algorítmicos con código, se especifica el comportamiento deseado con lenguaje natural. La diferencia es que el lenguaje natural admite ambigüedad, y esa ambigüedad debe gestionarse con cuidado.

Una consecuencia de esto es que el mismo modelo subyacente puede comportarse de forma radicalmente distinta según cómo se lo interpele. Un LLM al que se le pide 'explica la diferencia entre TCP (*Transmission Control Protocol*) y UDP (*User Datagram Protocol*)' producirá una respuesta muy distinta que si se le pide 'Eres un instructor de redes explicándole a un estudiante de primer año la diferencia entre TCP y UDP usando una analogía con el correo postal'. El contenido técnico puede ser similar, pero la forma, el nivel de abstracción y la utilidad práctica para el usuario objetivo son completamente distintos.

Zero-shot, one-shot y few-shot prompting

La forma más elemental de *prompting* es el *zero-shot*, que consiste en pedirle al modelo que haga algo sin mostrarle ningún ejemplo. Simplemente se describe la tarea y él usa lo que aprendió durante el preentrenamiento para resolverla. No hay contexto adicional, no hay guía, no hay demostración. Por ejemplo: *Clasifica el siguiente texto como positivo, negativo o neutro. El servicio fue horrible.*

El modelo recibe eso y, sin haber visto ningún caso previo en el *prompt*, produce una respuesta. Lo puede hacer porque durante el preentrenamiento procesó cantidades enormes de texto donde ese tipo de tarea aparece de formas muy variadas. Aprendió qué significa clasificar, qué significa sentimiento, qué palabras suelen asociarse con cada categoría. Cuando le llega el *prompt*, activa ese conocimiento y lo aplica.

Puede funcionar para tareas comunes y bien definidas como traducir un texto, resumir un párrafo, responder una pregunta con hechos, corregir gramática. Son tareas que el modelo ha visto tantas veces durante el entrenamiento que puede resolverlas sin necesitar orientación adicional.

El problema aparece en tres situaciones típicas:

1. Cuando la tarea es inusual o muy alejada de lo que el modelo encontró durante el entrenamiento.

2. Cuando el formato de salida tiene que ser exactamente de cierta manera, algo que el modelo no puede saber si nadie se lo muestra.

3. Cuando la terminología es especializada o el contexto es muy particular, como el vocabulario interno de una empresa o patrones propios de un dominio técnico específico.

En esos casos, el modelo puede entender la tarea en términos generales, pero errar en los detalles que más importan. Y ahí es donde los ejemplos marcan la diferencia.

El *one-shot prompting* incorpora un único ejemplo antes de la tarea real. No es mucho, pero ya le da al modelo una referencia concreta de qué se espera. El *few-shot prompting* incluye varios

ejemplos, típicamente entre 3 y 10, lo que le permite al modelo captar patrones más estables y manejar mayor variabilidad en los casos nuevos.

Murphy (2022) describe este mecanismo como aprendizaje en contexto. A diferencia de la optimización del modelo cuando se reentrena y sus pesos se actualizan, acá no ocurre nada de eso. Los pesos quedan exactamente igual y lo que cambia es que el modelo, al ver los ejemplos dentro del *prompt*, ajusta implícitamente cómo interpreta la tarea y qué tipo de respuesta produce. Cuando termina la conversación, ese ajuste desaparece. No aprendió nada de forma permanente, solo lo usó en ese momento.

En innovación y desarrollo de producto, es la técnica favorita para construir prototipos rápidos. Con cuatro o cinco pares de *input* y *output* deseado, el modelo replica el comportamiento esperado sin ningún entrenamiento adicional, lo que la convierte en una herramienta muy accesible para equipos que quieren testear ideas rápido.

Chain-of-thought prompting

> Una de las técnicas de *prompting* más poderosas descubiertas en los últimos años es el *chain-of-thought (CoT) prompting*. Parte de una observación bastante simple: para tareas que requieren razonamiento en varios pasos, pedirle al modelo que piense en voz alta antes de dar una respuesta mejora drásticamente su precisión.

Wei, Wang, Schuurmans, Bosma, Ichter, Xia, Chi, Le y Zhou (2022) demostraron que agregar la instrucción 'Pensemos paso a paso' a un *prompt* puede transformar el rendimiento en problemas de razonamiento aritmético y lógico. No hace falta rediseñar nada complejo. Solo cambiar cómo se le pide al modelo que responda.

La hipótesis sobre por qué funciona es la siguiente: generar el razonamiento intermedio ayuda al modelo a mantenerse en el camino correcto, en lugar de saltar directamente a una respuesta que suena válida pero es incorrecta. Cuando el modelo escribe cada paso, ese paso forma parte del contexto para el siguiente. Eso reduce la probabilidad de que cometa errores silenciosos que se acumulan sin que nadie los note.

Hay otra ventaja que desde el lado profesional importa mucho. El CoT hace auditable el proceso porque no se obtiene solo la respuesta, sino una traza de cómo se llegó a ella. Eso permite identificar exactamente en qué punto el razonamiento se desvía o incorpora una suposición cuestionable.

Wang et al. (2024) lo describen como un componente esencial en el diseño de agentes autónomos más capaces, precisamente porque ese razonamiento explícito permite verificar y depurar el proceso de toma de decisiones.

En la práctica, la diferencia se nota en cualquier dominio donde el razonamiento importa más que la velocidad. Un especialista en seguridad que usa un modelo para analizar una secuencia de eventos en un *log* obtiene resultados muy distintos según cómo plantea la pregunta. Si pregunta directamente: "¿Es esto un ataque?", el modelo produce una respuesta. Si en cambio le pide que razone explícitamente sobre cada evento, que explique qué sugiere el evento A, qué implica el evento B dado lo anterior, y así sucesivamente, el análisis es más sólido, más auditable, y si el modelo comete un error de razonamiento, el analista puede identificarlo y corregirlo en lugar de recibirlo empaquetado dentro de una conclusión opaca.

Principios fundamentales del diseño de prompts

Más allá de las categorías formales, el diseño de *prompts* efectivos es una práctica que combina principios generales con ajuste empírico. Los siguientes principios son los más documentados en la literatura. Russell y Norvig (2016) remarcan que la especificación precisa de los objetivos es fundamental en cualquier sistema de IA, y el *prompting* no es la excepción.

- **Claridad y especificidad:** instrucciones vagas producen respuestas vagas. En lugar de 'analiza este texto', es mejor 'identifica las tres afirmaciones principales del siguiente texto y evalúa si cada una está respaldada por evidencia explícita, usando el formato: Afirmación / Evidencia presente / Evaluación'.

- **Rol y contexto:** proporcionar al modelo un rol explícito y contexto suficiente sobre la tarea mejora consistentemente la calidad y pertinencia de las respuestas. 'Eres un experto en ciberseguridad revisando un informe de incidentes para una empresa financiera' produce respuestas muy distintas que el mismo *prompt* sin ese contexto, porque orienta al modelo

sobre qué vocabulario usar, qué nivel de detalle técnico es apropiado y qué aspectos priorizar.

- **Control del formato de salida:** especificar explícitamente el formato esperado es clave cuando la respuesta del modelo se convierte en fuente dentro del sistema. 'Responde en formato JSON con los campos entidad, tipo y relevancia' asegura que la salida pueda integrarse directamente con el resto del sistema sin necesidad de parseo complejo o corrección manual.

- **Iteración y prueba:** el primer *prompt* raramente es el mejor. El *prompting* efectivo es un proceso iterativo donde se prueba, se analiza dónde falla y se refina. El criterio siempre es la calidad de la respuesta para el caso específico, no la longitud del *prompt*. Un *zero-shot* bien especificado, muchas veces supera a un *few-shot* mal elegido.

Los modelos modernos como GPT-4, Claude o Gemini permiten separar las instrucciones generales del mensaje puntual que manda el usuario. Esa separación se hace entre el *system prompt* y los mensajes de usuario.

Se configura antes de que empiece la conversación, generalmente desde el *backend* de la aplicación o en el panel de configuración de la API. Es invisible para el usuario final, pero define todo el comportamiento del modelo: su rol, su tono, las restricciones que debe respetar, el formato en que debe responder. Los mensajes del usuario, en cambio, son las tareas concretas que llegan en tiempo real durante la conversación.

Esta separación es lo que permite construir aplicaciones consistentes. Sin ella, habría que repetir todas las instrucciones en cada mensaje, con el riesgo de que el modelo se comporta de forma distinta según cómo esté redactado cada uno.

En aplicaciones profesionales, el *system prompt* es donde se define la personalidad del asistente. Un asistente de análisis de seguridad, por ejemplo, podría tener instrucciones para siempre indicar el nivel de confianza de sus evaluaciones, nunca hacer afirmaciones categóricas sobre atribución de ataques y estructurar cada respuesta en secciones de hallazgos, evidencia y recomendaciones. El usuario no ve nada de eso, pero lo experimenta en cada interacción.

Bommasani et al. (2021) señalan que el diseño de esta interfaz de interacción, incluido el *system prompt*, es una de las decisiones más determinantes en el comportamiento observado de un *foundation model* en producción. No es un detalle técnico menor, sino donde se toman las decisiones de diseño que más impactan en cómo se comporta el modelo en el mundo real.

Uso de herramientas

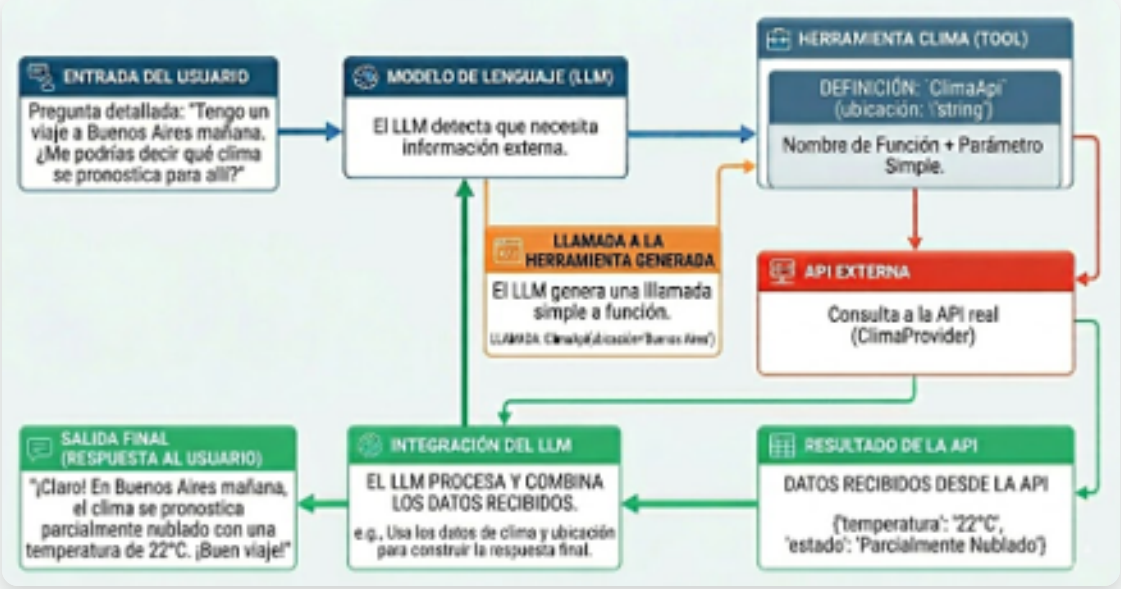
Un LLM puro, por potente que sea, enfrenta limitaciones, ya que su conocimiento está congelado en el momento del entrenamiento y no puede acceder a bases de datos propietarias. La solución a estas limitaciones no es esperar al próximo modelo más grande, sino equipar al modelo actual con herramientas externas que expandan su capacidad.

Wang et al. (2024) documentan cómo esta capacidad de uso de herramientas es uno de los pilares de los agentes autónomos basados en LLM. La idea conceptual es que el modelo actúa como el módulo de razonamiento que decide qué herramientas usar, cuándo usarlas y con qué parámetros; mientras que las herramientas aportan lo que el modelo solo no puede: datos actualizados, cómputo y acceso a sistemas externos.

API externas y herramientas

La forma más directa de extender lo que puede hacer un LLM es darle acceso a herramientas. No solo generar texto, sino generar llamadas a funciones específicas, recibir el resultado e integrarlo en la respuesta.

Figura 4. Generación de texto usando herramientas



Fuente: elaboración propia.

El flujo es bastante limpio porque el usuario hace una pregunta, el modelo detecta que necesita datos externos para responderla bien, arma una llamada estructurada a la herramienta correspondiente, recibe lo que devuelve y redacta una respuesta en lenguaje natural. El usuario final generalmente no ve ese proceso intermedio. Solo recibe una respuesta que incorpora información real y actualizada.

Vale aclarar cómo se relacionan las API con las herramientas, porque no son lo mismo. Una API es el canal técnico: un *endpoint*, parámetros, una respuesta. Una herramienta es una capa de abstracción por encima de eso, es decir, es como una API empaquetada con un nombre, una descripción de para qué sirve y un esquema de qué parámetros acepta. Lo que el modelo ve y decide invocar son las herramientas, no las API directamente. Por debajo siempre hay una API, pero el modelo opera a un nivel más alto.

Cuando se crea un sistema así, definimos las herramientas disponibles y el modelo elige en tiempo de inferencia cuál usar según lo que necesite. Como desarrollador, tu trabajo es controlar qué herramientas exponer, qué API se conectan de fondo y qué permisos otorgar a cada una.

Para estandarizar todo esto, Anthropic desarrolló MCP (*Model Context Protocol*), un protocolo abierto que define una forma común para que cualquier servicio externo se exponga como herramienta para Claude. En lugar de que cada integración se implemente a su manera, MCP establece el contrato. Servicios como Canva, Gmail o Notion lo implementan una vez y quedan

disponibles como herramientas que el modelo puede invocar directamente dentro de una conversación.

OpenAI y Google tienen sistemas equivalentes, los GPTs con *Actions* en el caso de ChatGPT y las extensiones de Workspace en el caso de Gemini. La lógica es la misma en todos: darle al modelo acceso a herramientas externas en tiempo de inferencia. La diferencia es que MCP apuesta a ser un estándar abierto, lo que en teoría permite que una integración funcione con cualquier modelo que soporte el protocolo.

Para entender el valor real de este enfoque, nada mejor que ver sus casos de uso. En ciencia de datos, un modelo conectado a la API de una base de datos puede responder basándose en la información real de la organización, en lugar de alucinar patrones estadísticos. En seguridad informática, permite correlacionar indicadores de compromiso con amenazas en tiempo real, algo inviable si dependiera solo del conocimiento estático de su entrenamiento. Y en bioinformática, la integración con repositorios como UniProt le permite razonar sobre información molecular en constante actualización, superando al conocimiento estático.

Generación aumentada por recuperación (RAG)

> Un modelo de lenguaje, cuando termina de entrenarse, queda estático. Todo lo que sabe lo aprendió hasta una fecha concreta, y a partir de ahí no entra nada nuevo. Si se consulta sobre un evento de la semana pasada, o sobre el reglamento interno de tu empresa, no tiene esos datos. Y como está diseñado para generar texto coherente, los inventa. Con impecable redacción, con total confianza, completamente falso. Eso se llama alucinación, y es el problema central que RAG viene a resolver.

La solución obvia parecía ser el *fine-tuning*. Si reentrenas el modelo con los datos nuevos, sin dudas resolvemos el problema. Sin embargo, este enfoque exige un costo de cómputo muy alto, requiere mano de obra especializada y, en cuanto el proceso finaliza, la información ya ha comenzado a quedar obsoleta. Es como imprimir una enciclopedia para resolver el dilema de la desactualización de los libros.

De hecho, como señala la documentación técnica de AWS (s. f.), reentrenar modelos fundacionales para incorporar datos específicos de una organización implica costos computacionales y financieros que resultan prohibitivos para la mayoría de los casos de uso reales.

RAG hace algo más inteligente, porque separa el razonamiento del almacenamiento. El modelo se queda haciendo lo que sabe hacer: leer, analizar y redactar. La información de hecho vive afuera, en una base de datos que se puede actualizar en cualquier momento sin tocar el modelo. Bommasani et al. (2021) refuerzan que reentrenar un *foundation model* completo cada vez que hay información nueva es computacionalmente prohibitivo, mientras que RAG permite extender el conocimiento del modelo de forma dinámica y eficiente.

¿Cómo funciona en la práctica? Primero, hay una fase de preparación donde se recopilan todos los documentos que el sistema necesita conocer. Estos archivos se dividen en fragmentos manejables y cada uno se procesa a través de un modelo de *embeddings*. Este modelo convierte el texto en un vector numérico de alta dimensionalidad que, en esencia, es una representación matemática de su significado; captura el concepto subyacente y no solo las palabras exactas. Finalmente, estos vectores se almacenan en una base de datos vectorial.

Cuando el usuario realiza una consulta, esta se convierte en un vector utilizando el mismo modelo. Luego, el sistema busca los fragmentos cuyos vectores están más próximos al de la pregunta. En esto consiste la búsqueda semántica: no rastrea palabras idénticas, sino significados similares. Si alguien pregunta "¿cuál es la política de vacaciones?" y el documento interno dice "días de descanso anuales remunerados", el sistema los asociará de todos modos.

AWS (s. f.) ilustra este principio con un ejemplo directo: un empleado que consulta sobre sus días de descanso activa una búsqueda que recupera tanto la política de la organización como su historial individual, dado que ambos documentos son semánticamente relevantes para la consulta.

Una vez recuperados los fragmentos relevantes, el sistema los empaqueta junto con la pregunta original y se lo manda todo al modelo. El modelo tiene los documentos delante; su trabajo es leerlos, extraer lo que responde la pregunta y redactarlo de forma clara.

A nivel operativo, esto cambia las reglas del juego. Actualizar una base de datos vectorial es trivial en comparación con reentrenar un modelo, lo que disminuye los costos. Por un lado, las alucinaciones se reducen al mínimo porque el modelo opera sobre información real y acotada. Por otro lado, si el sistema está bien configurado, es capaz de citar textualmente de qué documento

extrajo cada respuesta, lo que permite auditar el proceso. Incluso en términos de seguridad, se pueden gestionar permisos a nivel de recuperación, ya que el sistema solo indexa los fragmentos a los que el usuario tiene acceso, sin que el modelo deba intervenir en esa lógica.

RAG no vuelve al modelo más inteligente; simplemente le da los datos correctos en el momento oportuno. La inteligencia ya estaba ahí; lo que faltaba era contexto.

En seguridad informática, un sistema con RAG puede extraer reportes de incidentes similares al caso actual y entregárselos al modelo para que razone por analogía, sin necesidad de que esos registros hayan formado parte de su entrenamiento. En bioinformática, un asistente es capaz de recuperar artículos científicos recientes sobre una mutación específica para contextualizar su análisis, integrando de este modo investigaciones publicadas con posterioridad al corte de entrenamiento. Finalmente, en ciencia de datos, el sistema permite acceder a la documentación técnica interna, diccionarios de variables y procesos de negocio; activos de conocimiento que ningún modelo preentrenado posee y que resultan indispensables para interpretar correctamente la información de una organización particular.

Memoria externa y navegación web

Más allá de recuperar información de bases de datos, los LLM pueden equiparse con memoria externa que persiste entre conversaciones. Esto permite construir asistentes que recuerdan el historial de un usuario, las preferencias aprendidas en interacciones anteriores o el estado de un proyecto en curso. La limitación que se supera es estructural, ya que un LLM sin memoria olvida absolutamente todo al terminar una conversación.

La navegación web es otra extensión con mucho potencial. Un modelo con acceso a un navegador puede buscar información actualizada, leer artículos, completar formularios y extraer datos en tiempo real. Wang et al. (2024) documentan que esta capacidad es uno de los factores centrales para construir agentes autónomos genuinamente útiles en tareas del mundo real, donde es fundamental operar sobre información actualizada.

Agentes autónomos basados en LLM

Hasta ahora has visto los componentes por separado: un LLM que razona, herramientas que extienden lo que puede hacer, memoria que le da continuidad. Cuando estos elementos se

combinan en un sistema que opera con un objetivo dado y toma decisiones propias para alcanzarlo, te encuentras ante lo que se conoce como un agente autónomo basado en LLM.

Wang et al. (2024) los definen como sistemas que usan el LLM como módulo central de razonamiento para percibir el entorno, planificar una secuencia de acciones, ejecutarlas a través de herramientas y ajustar el plan según los resultados observados.

Lo que vale la pena notar es que esta definición no es nueva: Russell y Norvig (2016) describen exactamente este ciclo (percepción, razonamiento, acción) como la arquitectura fundamental de cualquier agente inteligente. Lo que cambia con los LLM no es la lógica del ciclo, sino la naturaleza del módulo de razonamiento y la amplitud de lo que puede manejar sin ingeniería especializada.

El ciclo operativo comienza cuando el agente recibe un objetivo en lenguaje natural, lo descompone en subtareas, ejecuta cada una usando las herramientas disponibles, evalúa qué obtuvo y decide si continúa, ajusta o concluye. No hay un *script* fijo que determine esa secuencia; el modelo infiere qué hacer en cada paso a partir del estado actual de la tarea.

En ciencia de datos, un agente puede recibir la instrucción: "Analiza el dataset sales_2024.csv, identifica los tres patrones más significativos y genera un informe en PDF con visualizaciones" y ejecutar esa cadena completa de forma autónoma: invocar herramientas de análisis, generar los gráficos, compilar el documento.

En seguridad informática, un agente de respuesta a incidentes puede recibir una alerta, investigar el contexto en múltiples sistemas, correlacionar con amenazas conocidas en bases y realizar un escalamiento si es necesario; todo sin intervención humana paso a paso. Un agente de investigación en bioinformática podría navegar por PubMed para identificar los artículos más relevantes sobre una proteína específica.

Lo que hace útiles a estos sistemas no es que sean más inteligentes que las herramientas que orquestan, sino que pueden razonar sobre cuándo y cómo combinarlas, sin que alguien tenga que especificar esa lógica de antemano para cada caso posible. Ese es el salto.

Aplicación al caso

Al comienzo de la lectura pudiste acceder al caso de CNET: textos fluidos, errores severos, editores que no los detectaron, rendimiento desigual según el tema. Cada uno de esos patrones parecía

desconcertante. Con los conceptos expuestos, todos deberían ser comprensibles.

El primer patrón, el error en el cálculo del interés compuesto presentado con total confianza, es la consecuencia más directa de cómo funciona la generación de texto en un LLM. El sistema no busca el resultado correcto en una base de datos ni verifica si sus afirmaciones son verdaderas. Para cada posición de la secuencia, calcula una distribución de probabilidades sobre el vocabulario completo y genera el *token* más probable dado el contexto.

Lo que se percibe como confianza es alta probabilidad estadística, no certeza factual. Un dato incorrecto puede tener la misma probabilidad que uno correcto si el contexto estadístico del *prompt* lo favorece. Eso explica por qué los errores son tan difíciles de detectar, ya que el sistema no tiene señales de error que emitir.

El segundo patrón, el rendimiento desigual según el tema, se explica por la combinación del entrenamiento y el fenómeno de emergencia. Los LLM aprenden los patrones estadísticos del lenguaje en la proporción en que ese lenguaje existe en sus datos de entrenamiento. Hay mucho más texto en inglés sobre finanzas personales generales que texto técnico detallado sobre productos específicos del mercado financiero estadounidense.

El fenómeno de emergencia documentado por Bommasani et al. (2021) agrega otra dimensión: ciertas capacidades no escalan de forma gradual: aparecen de golpe al cruzar umbrales de escala, y otras nunca aparecen independientemente del tamaño del modelo. Eso significa que la competencia en un dominio no predice la competencia en otro, y que el mismo modelo puede ser sólido en conceptos generales de finanzas y fallar en detalles específicos de productos sin que nadie lo haya diseñado así.

El tercer patrón, que los editores humanos no detectaran los errores, presenta algo sobre el mecanismo de atención. El modelo produce texto cuya estructura, vocabulario y ritmo son estadísticamente similares a los textos correctos sobre finanzas personales que procesó durante el entrenamiento. Eso hace que el texto suene autorizado. Y cuando algo suena autorizado, la lectura crítica se vuelve más difícil para el lector humano. No es un error del editor; es una consecuencia de cómo funciona el modelo y cómo los humanos leen texto bien redactado.

RAG aplicado al caso de CNET habría significado que, ante una consulta sobre tasas de interés, el sistema recuperarían primero documentos verificados como tablas de tasas reales, definiciones regulatorias, fichas técnicas de productos financieros; y razonaría sobre esa información en lugar

de inferir estadísticamente lo que podría ser correcto. El modelo no se vuelve más inteligente, pero recibe los datos correctos en el momento oportuno. Como señalan Bommasani et al. (2021), reentrenar el modelo completo cada vez que la información cambia es computacionalmente prohibitivo; RAG resuelve el problema de la actualización sin tocar los pesos del modelo.

Finalmente, la ventana de contexto establece el límite operativo concreto del sistema. En artículos cortos de finanzas personales, este límite no era el problema central. Pero en tareas más largas, como analizar contratos extensos o revisar un informe técnico de cien páginas, el modelo no puede mantener coherencia entre todas las partes si el documento supera la ventana disponible. No es un *bug*, es una consecuencia estructural de cómo funciona la atención en los *transformers*.

Lo que el caso de CNET demuestra es que implementar un LLM en producción exige entender qué tipo de sistema es. No una base de datos, no un motor de búsqueda: un generador de texto estadísticamente plausible cuya confianza aparente no indica certeza factual.

ACTIVIDADES DE REPASO Y REFUERZO

PREGUNTAS VERDADERO/FALSO

Pregunta: Los modelos fundacionales son sistemas de IA entrenados para una única tarea específica y no pueden adaptarse a otras sin ser completamente reentrenados desde cero.

- ○ Es verdadero, porque cada modelo de IA se especializa en un dominio y su conocimiento no puede transferirse a otros sin reentrenamiento completo.
- ○ Es falso, porque los modelos fundacionales están entrenados sobre volúmenes masivos de datos variados y se adaptan a una diversidad de tareas posteriores sin reentrenamiento completo.

REFERENCIAS

Amazon Web Services. (s. f.). ¿Qué es RAG (generación aumentada por recuperación)? https://aws.amazon.com/es/what-is/retrieval-augmented-generation/

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., Koreeda, Y., y Liang, P. (2021). *On the opportunities and risks of foundation models.* Center for Research on Foundation Models, Stanford University. https://arxiv.org/abs/2108.07258

Murphy, K. P. (2022). *Probabilistic machine learning: An introduction.* MIT Press. https://probml.github.io/pml-book/book1.html

Raschka, S., y Mirjalili, V. (2017). *Python machine learning: Machine learning and deep learning with Python, scikit-learn, and TensorFlow* (2.ª ed.). Packt Publishing.

Russell, S., y Norvig, P. (2016). *Artificial intelligence: A modern approach* (3.ª ed.). Pearson.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., y Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. https://arxiv.org/abs/1706.03762

Wang, L., Ma, C., Feng, X., Zhang, Z. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18, Artículo 186345. https://doi.org/10.1007/s11704-024-40231-1

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., y Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837. https://arxiv.org/abs/2201.11903